```tsx
import React, { useCallback, useMemo as useMemoReact } from "react";
import {
  ChevronLeft,
  ChevronRight,
  ChevronFirst,
  ChevronLast,
  MoreHorizontal,
} from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🚀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// 🛢 LOCAL TYPE ISOLATION GATEWAY
export interface PaginationItem {
  type: "page" | "ellipsis" | "first" | "last" | "prev" | "next";
  page?: number;
  label: string;
  ariaLabel: string;
  isActive?: boolean;
  isDisabled?: boolean;
}

export interface StandalonePaginationProps {
  /** Current active page (1-indexed) */
  currentPage: number;
  /** Total number of pages */
  totalPages: number;
  /** Number of sibling pages to show on each side of current page */
  siblingCount?: number;
  /** Number of pages to show at the start/end boundaries */
  boundaryCount?: number;
  /** Callback when page changes */
  onPageChange: (page: number) => void;
  /** Custom className for the pagination container */
  className?: string;
  /** Custom className for individual page items */
  itemClassName?: string;
  /** Custom className for active page item */
  activeItemClassName?: string;
  /** Custom className for disabled items */
  disabledItemClassName?: string;
  /** Show first/last page buttons */
  showFirstLast?: boolean;
  /** Show previous/next buttons */
  showPrevNext?: boolean;
  /** Custom labels */
```

```
  labels?: {
    first?: { label?: string; ariaLabel?: string };
    last?: { label?: string; ariaLabel?: string };
    previous?: { label?: string; ariaLabel?: string };
    next?: { label?: string; ariaLabel?: string };
    ellipsis?: string;
    page?: (page: number) => string;
    pageAriaLabel?: (page: number) => string;
  };
  /** ARIA label for the pagination nav */
  ariaLabel?: string;
  /** Disable all interactions */
  disabled?: boolean;
  /** Size variant */
  size?: "sm" | "md" | "lg";
  /** Variant style */
  variant?: "default" | "outline" | "ghost";
}

// 🎯 SIBLING WINDOW MATH ENGINE
function calculatePaginationItems(
  clampedPage: number,
  totalPages: number,
  siblingCount: number = 1,
  boundaryCount: number = 1,
): PaginationItem[] {
  const items: PaginationItem[] = [];

  // Calculate sibling range using pre-clamped page
  const leftSiblingStart = Math.max(2, clampedPage - siblingCount);
  const rightSiblingEnd = Math.min(totalPages - 1, clampedPage + siblingCount);

  // Calculate boundary ranges
  const leftBoundaryEnd = Math.min(boundaryCount, totalPages);
  const rightBoundaryStart = Math.max(totalPages - boundaryCount + 1, 1);

  // Build page numbers to show
  const pagesToShow = new Set<number>();

  // Add left boundary pages
  for (let i = 1; i <= leftBoundaryEnd; i++) {
    pagesToShow.add(i);
  }

  // Add sibling pages around current
  for (let i = leftSiblingStart; i <= rightSiblingEnd; i++) {
    if (i >= 1 && i <= totalPages) {
      pagesToShow.add(i);
    }
  }
```

```
  // Add right boundary pages
  for (let i = rightBoundaryStart; i <= totalPages; i++) {
    pagesToShow.add(i);
  }

  // Convert to sorted array
  const sortedPages = Array.from(pagesToShow).sort((a, b) => a - b);

  // Build final items with ellipsis injection
  let lastPage = 0;
  for (const page of sortedPages) {
    // Inject ellipsis if there's a gap
    if (lastPage > 0 && page > lastPage + 1) {
      items.push({
        type: "ellipsis",
        label: "…",
        ariaLabel: "More pages",
      });
    }
    items.push({
      type: "page",
      page,
      label: page.toString(),
      ariaLabel: `Page ${page}`,
      isActive: page === clampedPage,
    });
    lastPage = page;
  }

  return items;
}

// 🎨 SIZE VARIANT TOKENS
const SIZE_CLASSES = {
  sm: "h-8 px-2.5 text-xs gap-0.5",
  md: "h-10 px-3 text-sm gap-1",
  lg: "h-12 px-4 text-base gap-1.5",
} as const;

const ITEM_SIZE_CLASSES = {
  sm: "h-8 w-8 text-xs",
  md: "h-10 w-10 text-sm",
  lg: "h-12 w-12 text-base",
} as const;

const VARIANT_CLASSES = {
  default:
    "bg-background border-border hover:bg-accent hover:text-accent-foreground",
  outline:
```

```
    "bg-transparent border-border hover:bg-accent hover:text-accent-foreground
border",
  ghost: "bg-transparent hover:bg-accent hover:text-accent-foreground",
} as const;

const ACTIVE_VARIANT_CLASSES = {
  default:
    "bg-brand-primary text-white border-brand-primary hover:bg-brand-hover",
  outline: "bg-brand-primary text-white border-brand-primary",
  ghost: "bg-brand-primary text-white",
} as const;

const DISABLED_CLASSES = "opacity-50 pointer-events-none cursor-not-allowed";

// 🎯 MAIN COMPONENT
export function StandalonePagination({
  currentPage,
  totalPages,
  siblingCount = 1,
  boundaryCount = 1,
  onPageChange,
  className,
  itemClassName,
  activeItemClassName,
  disabledItemClassName,
  showFirstLast = true,
  showPrevNext = true,
  labels = {},
  ariaLabel = "Pagination",
  disabled = false,
  size = "md",
  variant = "default",
}: StandalonePaginationProps) {
  // Clamp current page
  const clampedPage = useMemoReact(
    () => Math.max(1, Math.min(currentPage, totalPages)),
    [currentPage, totalPages],
  );

  // Memoized pagination items calculation
  const paginationItems = useMemoReact(
    () =>
      calculatePaginationItems(
        clampedPage,
        totalPages,
        siblingCount,
        boundaryCount,
      ),
    [clampedPage, totalPages, siblingCount, boundaryCount],
  );
```

```
// Memoized click handler
const handlePageClick = useCallback(
  (page: number) => {
    if (
      !disabled &&
      page >= 1 &&
      page <= totalPages &&
      page !== clampedPage
    ) {
      onPageChange(page);
    }
  },
  [disabled, totalPages, clampedPage, onPageChange],
);

// Memoized keyboard handler
const handleKeyDown = useCallback(
  (event: React.KeyboardEvent, page?: number) => {
    if (disabled) return;

    let targetPage: number | null = null;

    switch (event.key) {
      case "Enter":
      case " ":
        event.preventDefault();
        if (page !== undefined) {
          targetPage = page;
        }
        break;
      case "ArrowLeft":
        event.preventDefault();
        targetPage = Math.max(1, clampedPage - 1);
        break;
      case "ArrowRight":
        event.preventDefault();
        targetPage = Math.min(totalPages, clampedPage + 1);
        break;
      case "Home":
        event.preventDefault();
        targetPage = 1;
        break;
      case "End":
        event.preventDefault();
        targetPage = totalPages;
        break;
    }

    if (targetPage !== null && targetPage !== clampedPage) {
```

```
        onPageChange(targetPage);
      }
    },
    [disabled, clampedPage, totalPages, onPageChange],
  );

  // Early return if no pagination needed
  if (totalPages <= 1) {
    return null;
  }

  const sizeClass = SIZE_CLASSES[size];
  const itemSizeClass = ITEM_SIZE_CLASSES[size];
  const variantClass = VARIANT_CLASSES[variant];
  const activeVariantClass = ACTIVE_VARIANT_CLASSES[variant];

  return (
    <nav
      role="navigation"
      aria-label={ariaLabel}
      className={cn(
        "flex items-center justify-center gap-1.5",
        "font-medium",
        className,
      )}
    >
      {/* First Page Button */}
      {showFirstLast && clampedPage > 1 && (
        <button
          type="button"
          onClick={() => handlePageClick(1)}
          onKeyDown={(e) => handleKeyDown(e, 1)}
          disabled={disabled || clampedPage === 1}
          className={cn(
            "inline-flex items-center justify-center",
            "rounded-md border",
            "font-medium transition-all duration-150 ease-out",
            "focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 focus-visible:ring-offset-background",
            "active:scale-[0.98]",
            sizeClass,
            variantClass,
            disabled && DISABLED_CLASSES,
            disabledItemClassName,
          )}
          aria-label={labels.first?.ariaLabel ?? "Go to first page"}
          title={labels.first?.label ?? "First page"}
        >
          <ChevronFirst
            size={size === "sm" ? 14 : size === "md" ? 16 : 18}
```

```
            strokeWidth={2.5}
            aria-hidden="true"
          />
        </button>
      )}

      {/* Previous Page Button */}
      {showPrevNext && clampedPage > 1 && (
        <button
          type="button"
          onClick={() => handlePageClick(clampedPage - 1)}
          onKeyDown={(e) => handleKeyDown(e, clampedPage - 1)}
          disabled={disabled || clampedPage === 1}
          className={cn(
            "inline-flex items-center justify-center",
            "rounded-md border",
            "font-medium transition-all duration-150 ease-out",
            "focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 focus-visible:ring-offset-background",
            "active:scale-[0.98]",
            sizeClass,
            variantClass,
            disabled && DISABLED_CLASSES,
            disabledItemClassName,
          )}
          aria-label={labels.previous?.ariaLabel ?? "Go to previous page"}
          title={labels.previous?.label ?? "Previous page"}
        >
          <ChevronLeft
            size={size === "sm" ? 14 : size === "md" ? 16 : 18}
            strokeWidth={2.5}
            aria-hidden="true"
          />
        </button>
      )}

      {/* Page Numbers & Ellipsis */}
      <div
        className="flex items-center gap-0.5"
        role="group"
        aria-label="Page numbers"
      >
        {paginationItems.map((item, index) => {
          const isActive = item.isActive;
          const isEllipsis = item.type === "ellipsis";

          if (isEllipsis) {
            return (
              <span
                key={`ellipsis-${index}`}
```

```
              className={cn(
                "inline-flex items-center justify-center",
                "text-muted-foreground",
                "select-none",
                itemSizeClass,
                itemClassName,
              )}
              aria-hidden="true"
            >
              <MoreHorizontal
                size={size === "sm" ? 14 : size === "md" ? 16 : 18}
                strokeWidth={2}
                aria-hidden="true"
              />
            </span>
          );
        }

        return (
          <button
            key={`page-${item.page}`}
            type="button"
            onClick={() => item.page && handlePageClick(item.page)}
            onKeyDown={(e) => handleKeyDown(e, item.page)}
            disabled={disabled || isActive}
            className={cn(
              "inline-flex items-center justify-center",
              "rounded-md border",
              "font-medium transition-all duration-150 ease-out",
              "focus-visible:outline-none focus-visible:ring-2
focus-visible:ring-ring focus-visible:ring-offset-2
focus-visible:ring-offset-background",
              "active:scale-[0.98]",
              itemSizeClass,
              isActive ? activeVariantClass : variantClass,
              isActive && "cursor-default",
              disabled && DISABLED_CLASSES,
              isActive ? activeItemClassName : itemClassName,
            )}
            aria-label={item.ariaLabel}
            aria-current={isActive ? "page" : undefined}
            tabIndex={isActive ? 0 : -1}
          >
            {labels.page ? labels.page(item.page!) : item.label}
          </button>
        );
      })}
    </div>

    {/* Next Page Button */}
```

```
      {showPrevNext && clampedPage < totalPages && (
        <button
          type="button"
          onClick={() => handlePageClick(clampedPage + 1)}
          onKeyDown={(e) => handleKeyDown(e, clampedPage + 1)}
          disabled={disabled || clampedPage === totalPages}
          className={cn(
            "inline-flex items-center justify-center",
            "rounded-md border",
            "font-medium transition-all duration-150 ease-out",
            "focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 focus-visible:ring-offset-background",
            "active:scale-[0.98]",
            sizeClass,
            variantClass,
            disabled && DISABLED_CLASSES,
            disabledItemClassName,
          )}
          aria-label={labels.next?.ariaLabel ?? "Go to next page"}
          title={labels.next?.label ?? "Next page"}
        >
          <ChevronRight
            size={size === "sm" ? 14 : size === "md" ? 16 : 18}
            strokeWidth={2.5}
            aria-hidden="true"
          />
        </button>
      )}

      {/* Last Page Button */}
      {showFirstLast && clampedPage < totalPages && (
        <button
          type="button"
          onClick={() => handlePageClick(totalPages)}
          onKeyDown={(e) => handleKeyDown(e, totalPages)}
          disabled={disabled || clampedPage === totalPages}
          className={cn(
            "inline-flex items-center justify-center",
            "rounded-md border",
            "font-medium transition-all duration-150 ease-out",
            "focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 focus-visible:ring-offset-background",
            "active:scale-[0.98]",
            sizeClass,
            variantClass,
            disabled && DISABLED_CLASSES,
            disabledItemClassName,
          )}
          aria-label={labels.last?.ariaLabel ?? "Go to last page"}
          title={labels.last?.label ?? "Last page"}
```

```
        >
          <ChevronLast
            size={size === "sm" ? 14 : size === "md" ? 16 : 18}
            strokeWidth={2.5}
            aria-hidden="true"
          />
        </button>
      )}
    </nav>
  );
}

export { StandalonePagination as Pagination };

export default StandalonePagination;
```